

Holdfast AR: Technical Documentation

Mobile AR survival shooter built with Unity 6 and AR Foundation. By **Joshua Chukwuebuka Moses**.
Repository: github.com/JOSHUALU/holdfast-ar

Holdfast AR is a first person survival shooter for phones. You play the last colonist on a planet called Kepler-9. You scan your real floor, drop your landing dome on it, and hold off three kinds of alien attackers with a ray gun until the evac ship arrives. If the timer runs out, you win. If your health hits zero, the dome falls.

1. Architecture overview

Everything runs in one scene. A **GameManager** owns a small state machine that moves between four states: Main Menu, Placement, Playing and Game Over. Each state sets itself up when it starts and cleans up when it ends, so leaving Playing always wipes every enemy and bullet. Gameplay code never calls the UI. It raises events on a static **GameEvents** class and the HUD listens, which meant I could rebuild the whole UI later without touching any combat code.

The UI is built from code by **UIManager** inside a 1080 by 1920 portrait frame that fits the phone's safe area. Every screen uses the same theme and sprites, which kept the layout consistent on different phone sizes.

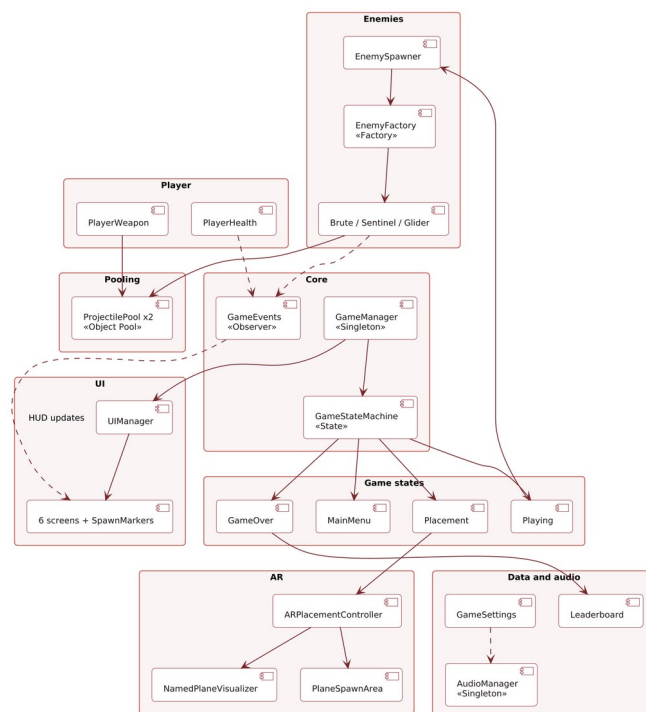


Figure 1. How the main systems connect (PlantUML).

2. OOP structure

Principle	Where I used it
Encapsulation	Data sits in private fields behind read only properties. Score only changes through <code>GameSession.RegisterKill()</code> , health only through <code>PlayerHealth.TakeDamage()</code> , and <code>Leaderboard</code> and <code>GameSettings</code> hide how they save to <code>PlayerPrefs</code> . Inspector values use <code>[SerializeField] private</code> .
Abstraction	<code>IDamageable</code> lets bullets and attacks hurt anything without knowing what it is. <code>IPoolable</code> gives pooled objects their reset hooks. <code>Enemy</code> , <code>GameState</code> and <code>UIPanel</code> are abstract base classes that set the rules without the details.
Inheritance	<code>MeleeEnemy</code> , <code>ShooterEnemy</code> and <code>DiverEnemy</code> all extend <code>Enemy</code> , which already handles health, chasing, turning, cooldowns, hit feedback and death. Four states extend <code>GameState</code> , six screens extend <code>UIPanel</code> , and <code>GameManager</code> and <code>AudioManager</code> extend <code>Singleton<T></code> .

Principle	Where I used it
Polymorphism	Enemy.Update() calls the abstract StoppingDistance and Attack(). The Brute walks up and punches, the Sentinel stops at 1.4 m and shoots, and the Glider overrides Animate() to fly and dive at the phone. The state machine only ever calls Enter, Tick and Exit, and UIManager only calls Show and Hide.

The three enemies

	Brute (melee)	Sentinel (shooter)	Glider (diver)
Model	Green alien, animated	Yellow bug mech, animated	Purple flying blob, animated
Bullets to kill	3	5	1
Behaviour	Walks up, stops at 0.35 m	Stops at 1.4 m	Flies in and dives at you
Attack	Punch within 0.5 m	Projectile, range 2.2 m	Bursts on impact (0.22 m)
Damage / cooldown	15 every 1.1 s	8 per shot every 1.6 s	20, once
Score	100	150	75

3. Design patterns used and why

Pattern	Classes	Why I used it
Object Pool	ObjectPool<T>, ProjectilePool, Projectile	The gun fires six shots a second. Creating and destroying bullets that often causes garbage collection spikes, and on a phone that shows up as stutter, which also hurts AR tracking.
State	GameStateMachine, GameState + 4 states	Each phase owns its own setup and cleanup. No giant switch statement, and the Start, Play, End flow is easy to follow.
Singleton	GameManager, AudioManager	There should only ever be one game coordinator and one sound owner, and both need to be easy to reach. Duplicates destroy themselves.
Factory	EnemyFactory	One place turns an enemy type into a prefab and hands it what it needs. Adding the Glider took no changes to the spawner logic beyond picking the type.
Observer	GameEvents, HudPanel, SpawnMarkers, GameSettings	Health, score, kills, time and spawns are broadcast as events. The HUD subscribes when it opens and unsubscribes when it closes. Settings changes tell AudioManager to update the music at once.

4. Object Pool implementation

- **Pre-initialised.** When the game loads, ProjectilePool creates 40 player bullets and 30 enemy bullets, switches them off and stores them in a stack. That is the only time a bullet is ever instantiated.
- **Getting a bullet.** Firing pops one off the stack, places it, switches it on and calls OnTakenFromPool(). If the stack is ever empty, the oldest bullet in flight gets recycled instead of creating a new one.
- **Returning it.** When a bullet hits something or reaches its 3 second lifetime, it hands itself back. The pool calls OnReturnedToPool(), switches it off and pushes it back on the stack. Double returns are ignored.
- **Proper reset.** Age, speed, damage, direction and team are cleared, and the trail is wiped so it doesn't draw a streak from its old position. At the end of a match ReleaseAll() pulls every bullet back.
- **Hit detection.** Each frame a bullet sphere-casts along its path, so fast bullets can't skip through small targets. It only damages things on the other team.

5. Sound system structure and sources

AudioManager is the only object in the game with AudioSources, so enemies and bullets don't each carry their own. It has one 2D source for player and UI sounds (using PlayOneShot, so rapid shots overlap cleanly), one looping music source, and a fixed set of six 3D sources that move to wherever an enemy sound happens. Clips load by name from Resources/Audio, each with its own volume and a slight

random pitch so repeats don't sound robotic. The Settings screen lets the player set sound effects and music to off, low or high.

Required event	Sound	Triggered in	Type
Player shoot	PlayerShoot	<code>PlayerWeapon.Fire()</code>	2D
Player death	PlayerDeath	<code>PlayerHealth.TakeDamage()</code> at 0 HP	2D
Enemy spawn	EnemySpawn	<code>EnemySpawner.SpawnOne()</code>	3D
Enemy shoot (Sentinel)	EnemyShoot	<code>ShooterEnemy.Attack()</code>	3D
Enemy damage (melee attack)	MeleeAttack	<code>MeleeEnemy.Attack()</code>	3D
Extra	PlayerHurt, EnemyHit, EnemyDeath, UIClick, Place, Victory, Ambient	Health, enemies, buttons, placement, end screen, menu	2D / 3D

Sources: I made all 12 sounds myself with a small Python script (`Tools/make_audio.py`) that builds them from sine, square and saw waves, pitch sweeps and filtered noise, saved as 16-bit WAV files. The ambient loop only uses frequencies that finish whole cycles in 16 seconds, so it loops without a click.

6. AR setup

- **Plane detection.** The plane manager only looks for horizontal planes.
- **Custom plane tracker.** I replaced Unity's default plane look with my own `NamedPlaneVisualizer`. It builds a mesh from each plane's outline and tiles a texture across it with **JOSHUA CHUKWUEBUKA MOSES** written on it. It only shows while a plane is being tracked.
- **Tap to place.** A tap raycasts against the planes and attaches an anchor where it hits, and the dome is parented to that anchor. After that, extra taps do nothing, plane detection stops and the trackers hide.
- **Spawning on planes.** When the dome lands, I save every plane's outline. Enemies only spawn at points that sit inside a real plane, 0.9 to 2.2 m from the dome and at least 1.1 m from the player. A small marker on the HUD shows where each one just appeared.
- **Leaderboard.** Each match is saved as JSON in `PlayerPrefs`. The newest goes first and the list is cut to the latest 5, so it survives closing the app.

7. Performance metrics

Measured in the Unity 6 editor with XR Simulation at 1080 by 1920, with six enemies on screen and the fire button held down for 15 seconds straight.

Metric	Result	What it shows
Average frame rate	57 FPS (editor capped near 60)	Smooth with a full wave and nonstop shooting
Frame time / render time	about 17 ms / 2.3 ms	Rendering is a small part of each frame
Draw calls / SetPass calls	56 / 14	Low enough for mid range phones
Triangles on screen	about 30,000	Low poly models keep the scene light
Garbage collections while firing	0 in 15 seconds	The pool stops bullet spam from creating garbage
Bullet objects in the scene	70 before, 70 after	Nothing is created or destroyed during play
Android APK size	45 MB	

Appendix: Difficulty and asset sources

Difficulty levels (bonus)

	Easy	Normal	Hard
Match length	60 s	90 s	120 s
Player health	150	100	80
Spawn every	3.2 s	2.4 s	1.6 s
Max enemies at once	4	6	9
Sentinel / Glider chance	25% / 15%	35% / 20%	40% / 25%
Enemy speed / damage	x0.8 / x0.6	x1 / x1	x1.3 / x1.4
Score multiplier	x0.8	x1	x1.5

Each level is a `DifficultySettings` ScriptableObject, so balancing never needs a code change.

Assets and sources

Asset	Used for	Source and licence
Ultimate Space Kit by Quaternius	Brute, Sentinel, Glider, dome, rocks, plants, solar panel	poly.pizza, CC0
Sci-Fi Gun Pack by Quaternius	Player ray gun	poly.pizza, CC0
UI Pack (Space Expansion) by Kenney	Panels, buttons, bars, crosshair, spawn markers	kenney.nl, CC0
Kenney Future font	All UI text and the plane tracker label	kenney.nl, CC0
Sounds, plane texture, app icon, animators	Made by me for this project	Original

Appendix: Diagrams

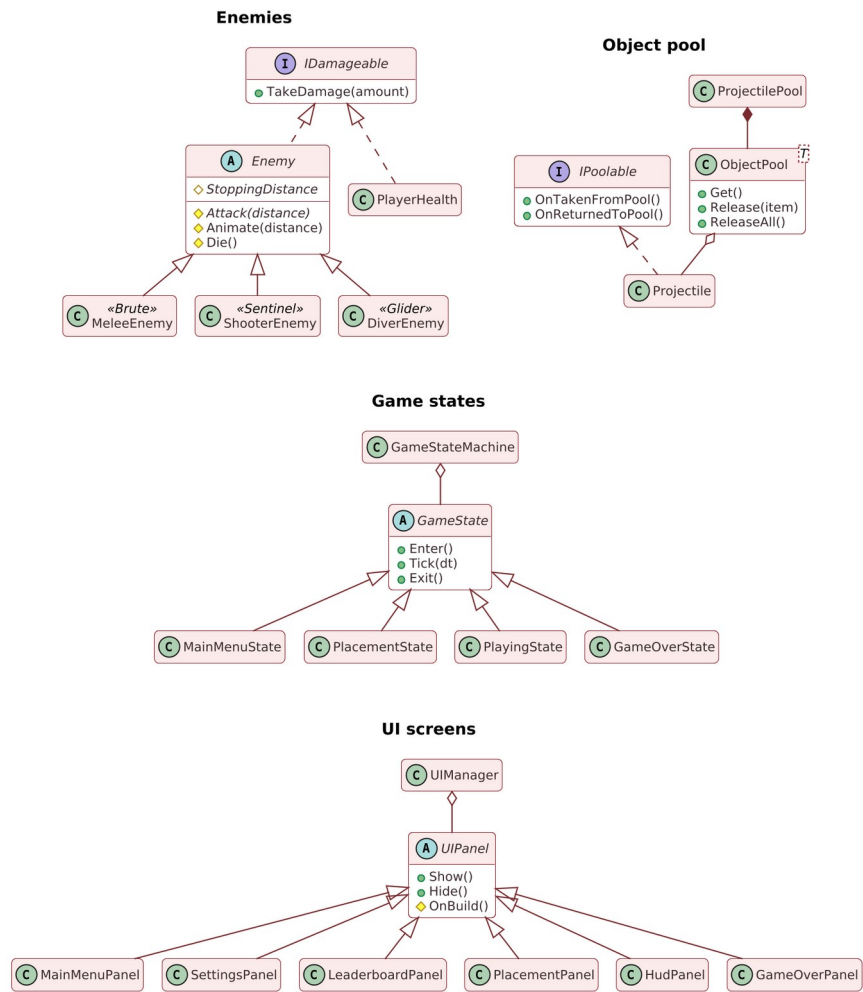


Figure 2. Class structure for enemies, the object pool, game states and UI screens.

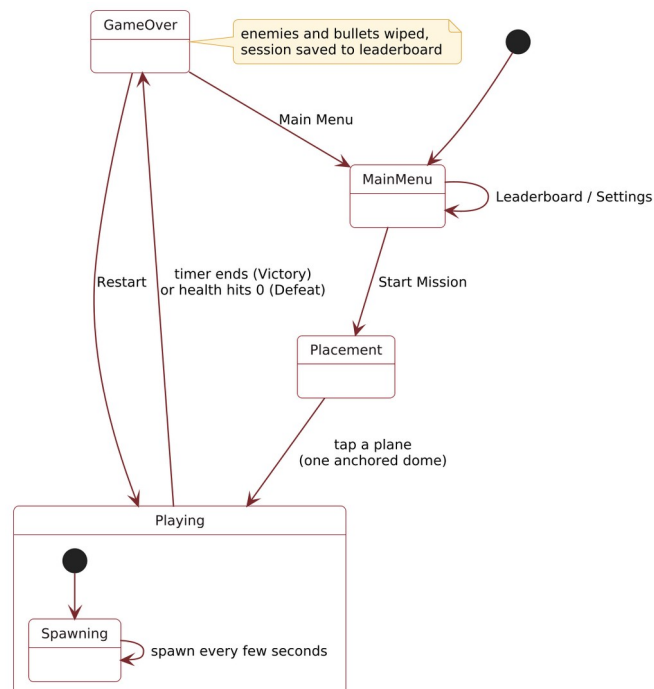


Figure 3. Game state flow from Start to Play to End.

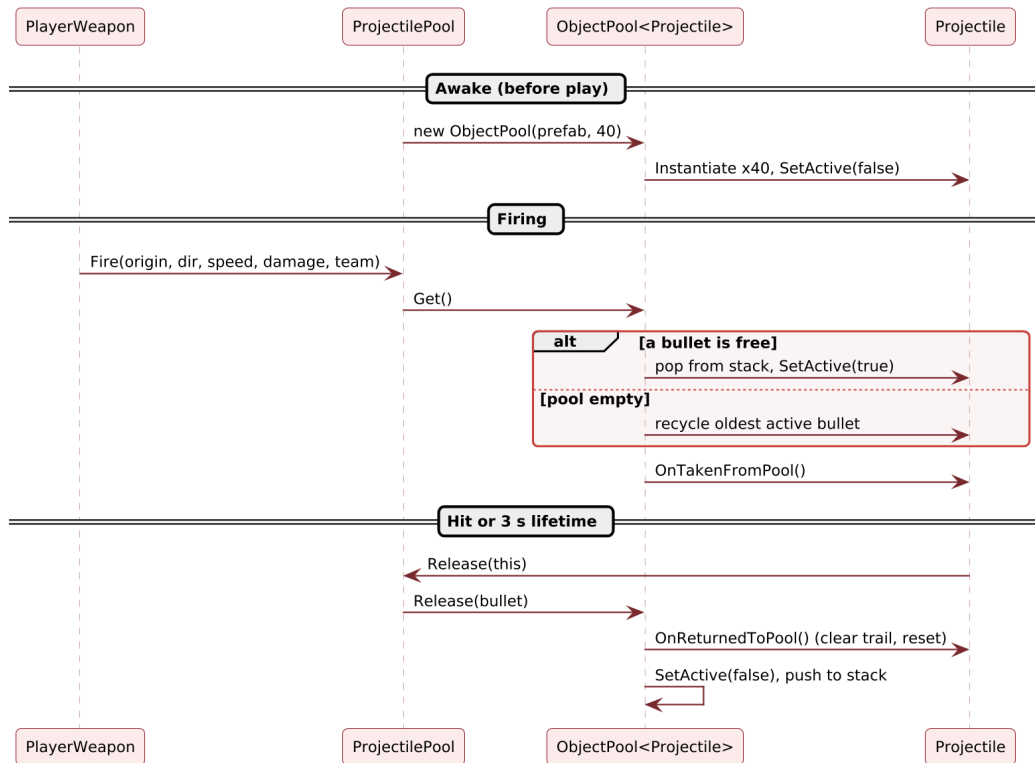
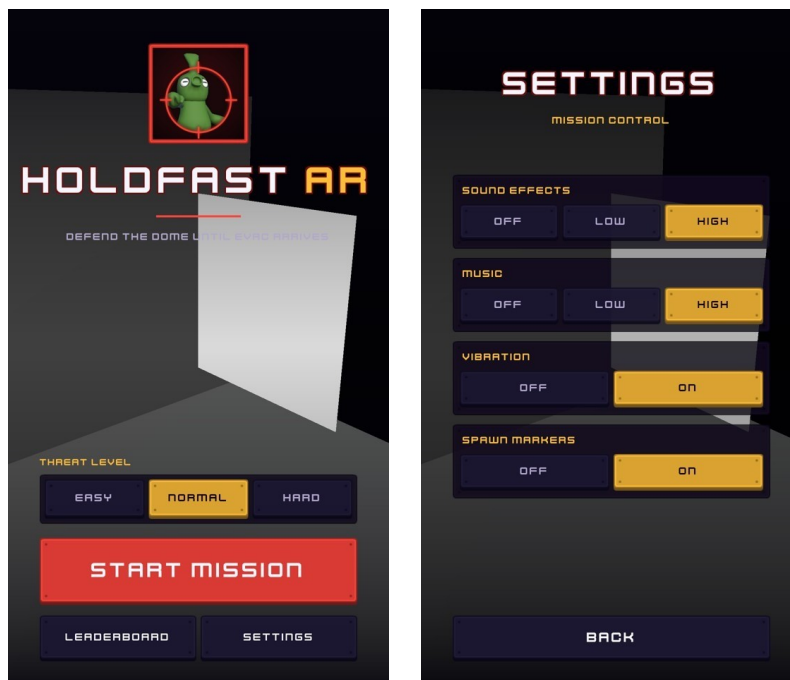


Figure 4. What happens when a bullet is fired and returned to the pool.

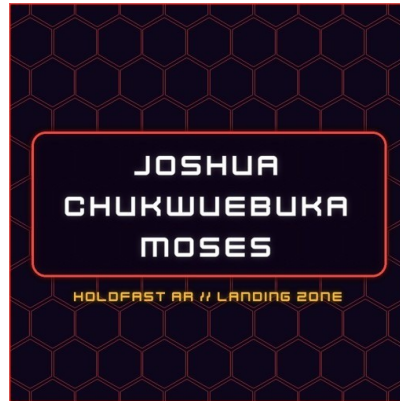
Appendix: Screenshots



Main menu and settings screen (XR Simulation room in the background).



The Brute, the Sentinel and the Glider side by side.



Custom plane tracker texture with my full name.